HATHOR-CORE · MASTER-BOOK

Settings & Configuration

How a node knows which network it is on and the hundreds of constants that define that network — `hathor/conf`, the profiles, and the `HathorSettings` object every component reads.

SUBJECT	hathor-core · Part II · the node, end to end
CHAPTER	22 · Part II · The Node
BRANCH	study/hathor-core-studies
EDITION	2026 · v1

YAML · Pydantic · network profiles · genesis · singleton accessor · read-only config · environment variables

Table of Contents

Chapter 22 — Settings & Configuration

What you'll learn in this chapter

- Why a full node needs hundreds of agreed-upon constants, and why getting even one of them wrong puts the node on a different network — not a broken one, a different one.
- What a **network profile** is (`mainnet`, `testnet`, a private net), and why Hathor stores profiles as **YAML** files rather than hard-coded Python.
- How `get_global_settings()` selects a profile from an environment variable, validates it into a `HathorSettings` object with **Pydantic**, and caches it as a process-wide **singleton**.
- Why the settings object is built **once, validated at load, and treated as read-only** — the fail-fast vs. crash-deep trade-off.
- Where settings sits in the boot sequence: right after the command line is parsed (Ch. 21), feeding the builder (Ch. 24) and then every component below it.

In Chapter 21 the command line handed control to `run_node`. The very next thing the node must establish, before it builds a single component, is which network it is joining and what the rules of that network are. That is this chapter. The package is small — four files in `hathor/conf/` — but it is load-bearing: nearly every other package reads from the object this one produces.

22.1 Localization — where `hathor/conf/` sits

`hathor/conf/` is **infrastructure** (the bottom band of the module map in Chapter 0, §0.4). It has almost no dependencies of its own and is depended on by almost everything else. It is one of the first things touched at boot and is then read for the lifetime of the process.

```
hathor-core/
├── hathor_cli/                ← parses the command line (Ch. 21) ─┐
│                                                                  │ sets an
└── hathor/                                                        │ env var,
    │   manager.py              ← reads settings constantly         │ then asks
    │                                                              ▼ conf for
    ├── conf/                  ◀ YOU ARE HERE                      the object
    │   ├── __init__.py        ← re-exports HathorSettings
    │   ├── get_settings.py    ← the accessor + env-var selection + singleton cache
    │   ├── settings.py        ← the HathorSettings type (Pydantic model + validators)
    │   └── mainnet.py         ← a legacy Python profile (being phased out)
    │
    ├── builder/               ← Ch. 24; receives the settings object, wires the node
    ├── transaction/           ← reads genesis, weights, token rules from settings
    ├── consensus/             ← reads weight/voiding params from settings
    ├── p2p/                   ← reads network name, ports, protocol params
    └── …                      ← essentially every package reads settings
```

The YAML profiles themselves (`mainnet.yml`, `testnet.yml`) do **not** live in this folder. They ship inside an installed dependency, `hathorlib`, and `hathor/conf/` points at them by default. We will come back to why in §22.5; for now, hold the shape: `hathor/conf/` is the machinery that loads a profile, and a profile is a file of constants.

> **Where this fits.** `hathor/conf/` answers one question for the rest of the node: "what are the rules of the network I am on?" It is read at boot by the builder and then continuously at runtime by verification, consensus, p2p, mining, and more. Because every component reads the same object, the package is also the node's single source of truth — there is no second place a constant can hide.

22.2 What it does and why it exists

A full node does not get to invent the rules. To be on the same network as everybody else, it must agree, to the bit, on a long list of constants:

- the **genesis**[1] — the exact first block and initial transactions, byte-for-byte;
- the **network name** (`mainnet`, `testnet-...`) used in the peer handshake;
- monetary rules — how many tokens genesis mints, the decimal places, the block reward schedule, the halving interval;
- timing and difficulty parameters — target seconds between blocks, the weight[2] math;
- the **feature-activation**[3] schedule — which protocol upgrades switch on, and when;
- structural limits — maximum transaction size, maximum number of inputs/outputs, script size caps;

- the network ports and bootstrap peers used to find the rest of the network.

There are hundreds of these. Here is the uncomfortable property that makes configuration a correctness concern and not a convenience: **a node with the wrong constants is not a broken node — it is a node on a different network.** If your node believes genesis is one set of bytes and the rest of the network believes another, the two will never share a common root of history (§0.3), so they can never agree on anything. Your node will reject every block the network sends and the network will reject every block yours sends. Nothing crashes. It just silently fails to be part of the network. The same is true, more subtly, of a single mismatched weight constant or a wrong reward schedule: your node will compute different validity and quietly fork off on its own.

So the job of `hathor/conf/` is narrow and strict:

1. **Decide which profile to load** — `mainnet`, `testnet`, or a private network — from the operator's choice.
2. **Load and validate** that profile into a single typed object, failing immediately if anything is malformed.
3. **Hand that one object to everyone**, and make sure everyone gets the same one.

The rest of this chapter is those three steps in code.

Why one read-only object, and not scattered constants

Imagine the naïve alternative: sprinkle the constants through the code as module-level globals — `MAX_TX_SIZE = 100_000` here, `DECIMAL_PLACES = 2` there. Three problems appear immediately.

You cannot switch networks. The constants are baked in at import time. Running the same binary against `testnet` would mean a different build. Hathor needs one binary that can join any network depending on a flag.

You cannot validate the set as a whole. Some constants constrain each other — for example, a proof-of-authority[4] network must not define block rewards (rewards are a proof-of-work concept). Scattered globals give you nowhere to check that rule. A single object validated at load gives you exactly one place.

You lose a single source of truth. With globals, a constant can be shadowed, re-assigned, or read inconsistently from two places. Bundling them into one object that is built once and never mutated means every component is reading the identical value. This is the **single source of truth** principle, and for network constants it is not a nicety — it is the difference between being on the network and not.

So Hathor uses one object, the `HathorSettings`, built once and treated as read-only. The next two sections give you the two background ideas the implementation rests on — the singleton and validated models — then §22.4 walks the code.

22.3 The concepts it rests on

This package is small precisely because it leans on two patterns you have already met. Here they are, re-established in the configuration context.

> **Recap — singleton / single global accessor (full treatment in Ch. 3).** A singleton is a value of which there is exactly one instance for the whole program, reached through a shared accessor rather than constructed by each caller. Configuration is the textbook case: every component wants "the settings," and they must all get the same settings, so you build the object once, stash it in a module-level variable, and have every caller go through one function — `get_global_settings()`. Chapter 3 flagged the singleton as a global-state smell to use sparingly; configuration is one of the few cases where it earns its place, because the alternative (threading a settings argument through every constructor in the codebase) is worse, and because the value is read-only after load.

> **Recap — Pydantic validation models (full treatment in Ch. 18).** Pydantic is a library for declaring a data shape as a Python class with type-annotated fields, and having it **validate and coerce** raw input (a dict parsed from YAML, say) into a typed object — rejecting anything that does not fit. You write `class HathorSettings(...)` with fields like `DECIMAL_PLACES: int`, and Pydantic guarantees that what you get back has an `int` there or raises an error trying. It also runs custom validators — small methods that enforce cross-field rules. Chapter 18 is the full treatment; here it is the mechanism that turns a YAML file into a trustworthy `HathorSettings`.

Read-only config, and why. A read-only[5] object is one nothing mutates after it is created. Settings are loaded once at boot and then read — never written — for the rest of the node's life. Treating them as read-only buys two things. First, safety: no stray line, anywhere in tens of thousands of lines, should quietly mutate a network constant mid-run and fork the node. This is the same invariant-protection idea from Chapter 1 — bundle the data and forbid casual mutation — applied to the most safety-critical data the node holds. Second, shareability: because nobody changes it, the one shared instance can be handed to every component without fear that one component's edit leaks into another's view. (The model also forbids unknown fields at load, which is the validation half of that safety story; we see exactly how in §22.4.)

> **Recap — genesis (from the Ch. 0 footnote; full treatment in Ch. 25 & 32).** The genesis is the hard-coded starting point of the ledger: the first block and the initial transactions, identical for every node on a network by definition. It is the shared root every node measures history from. It lives in the settings profile because it is a per-network constant — `mainnet` and `testnet` have different genesis data, and that difference is exactly what makes them different networks. We meet genesis as code in the vertex model (Ch. 25) and see how consensus anchors to it (Ch. 32); here, it is one (very consequential) entry in the profile.

22.4 The code, walked

The shape of the package

Three files do the work, and one re-exports.

```
hathor/conf/__init__.py     →  re-exports HathorSettings so callers can write
                               `from hathor.conf import HathorSettings`
hathor/conf/get_settings.py →  the accessor: choose a profile, load it, cache it
hathor/conf/settings.py     →  the HathorSettings type: a Pydantic model + validators
hathor/conf/mainnet.py      →  a legacy Python-module profile (deprecated path)
```

`__init__.py` is one line of substance — it re-exports the accessor so the rest of the codebase has a short, stable import:

```python
# hathor/conf/__init__.py:15
from hathor.conf.get_settings import HathorSettings
```

A toy first: a read-only config every function reads

Before the real code, here is the whole idea in a handful of lines of plain Python — a config built once, frozen against mutation, and reached through one accessor:

```python
from dataclasses import dataclass

@dataclass(frozen=True)              # frozen=True → assigning to a field raises
class Config:
    network: str
    max_tx_size: int

_config = None                       # the single cached instance (the singleton)

def get_config() -> Config:
    global _config
    if _config is None:              # build it the first time…
        _config = Config(network="mainnet", max_tx_size=100_000)
    return _config                   # …and hand back the same object every time after
```

Every function anywhere calls `get_config()` and gets the identical object. Try `get_config().max_tx_size = 5` and Python raises, because the dataclass is `frozen`. Hold this picture — Hathor's version is the same skeleton with two upgrades: the values come from a YAML file chosen at runtime, and validation is done by Pydantic instead of a bare dataclass.

Selecting the profile from the environment

The accessor lives in `get_settings.py`. The public entry point is `get_global_settings()`, and it does nothing but delegate:

```
# hathor/conf/get_settings.py:39
def get_global_settings() -> 'Settings':
    return HathorSettings()
```

`HathorSettings()` here is **a function, not the class** — a naming choice worth pausing on, because it is confusing the first time. The type `HathorSettings` lives in `settings.py`; this function in `get_settings.py` has the same name and returns an instance of that type. Most callers across the codebase use `get_global_settings()`; the same-named function is the historical spelling kept for compatibility. Both reach the same cache.

The function's job is to decide which profile to load, and it does so from environment variables[7]:

```
# hathor/conf/get_settings.py:43
def HathorSettings() -> 'Settings':
    settings_module_filepath = os.environ.get('HATHOR_CONFIG_FILE')
    if settings_module_filepath is not None:
        return _load_settings_singleton(settings_module_filepath, is_yaml=False)

    from hathorlib import conf
    settings_yaml_filepath = os.environ.get('HATHOR_CONFIG_YAML', conf.MAINNET_SETTINGS_FILEPATH)
    return _load_settings_singleton(settings_yaml_filepath, is_yaml=True)
```

Read this as a two-tier choice with a default:

1. If `HATHOR_CONFIG_FILE` is set, load a **Python-module** profile from that path (`is_yaml=False`). This is the legacy path — the docstring and a runtime warning (below) say it is being deprecated.
2. Otherwise, load a **YAML** profile from `HATHOR_CONFIG_YAML` (`is_yaml=True`).
3. If that env var is unset, fall back to `conf.MAINNET_SETTINGS_FILEPATH` — the `mainnet.yml` that ships inside the `hathorlib` package. **The default, with no configuration at all, is mainnet** (`get_settings.py:58`).

How does the env var get set? The operator does not normally set it by hand. Recall from Chapter 21 that `run_node` parses flags like `--testnet` or `--config-yaml <path>`; the CLI translates that choice into the `HATHOR_CONFIG_YAML` environment variable before anything calls `get_global_settings()`. So the data flow is: **flag → env var (set by the**

CLI) → profile path → loaded object. The environment variable is the hand-off channel between the command-line layer and the configuration layer; it keeps `hathor/conf/` from having to know anything about argument parsing.

> Why an environment variable and not a direct function argument? Because `get_global_settings()` is called from hundreds of places, most of which have no access to the parsed CLI object. An env var is a process-global channel that any of those call sites can read without being handed anything. It is the same reasoning as the singleton itself: the value is needed everywhere, so it travels through a global channel rather than down every call path.

Loading once, caching forever — the singleton

The actual load-and-cache lives in `_load_settings_singleton`. The module holds one cache slot:

```python
# hathor/conf/get_settings.py:30
class _SettingsMetadata(NamedTuple):
    source: str          # which file we loaded from
    is_yaml: bool        # YAML profile or legacy Python module?
    settings: 'Settings' # the validated HathorSettings instance

_settings_singleton: Optional[_SettingsMetadata] = None
```

`_settings_singleton` starts as `None` and is filled on first load. Note what is cached: not just the settings object, but also where it came from (`source`) and how (`is_yaml`). That bookkeeping powers a guard against a dangerous mistake — loading two different configurations into one process:

```python
# hathor/conf/get_settings.py:72
def _load_settings_singleton(source: str, *, is_yaml: bool) -> 'Settings':
    global _settings_singleton

    if _settings_singleton is not None:
        if _settings_singleton.is_yaml != is_yaml:
            raise Exception('loading config twice with a different file type')
        if _settings_singleton.source != source:
            raise Exception('loading config twice with a different file')
        return _settings_singleton.settings   # already loaded → hand back the same object
    # …first-load path below…
```

The logic: if the cache is already populated, return it — unless the caller is asking for a **different** file or file-type than the one already loaded, in which case raise. This enforces the single-source-of-truth invariant at runtime. A program that tried to be on two networks at once (load `mainnet` then `testnet`) is a bug so severe the package refuses to let it happen quietly; it crashes instead. This is the singleton pattern doing its second job: not just "build once" but "guarantee everyone agrees."

On the first load, the function warns if the legacy Python-module path was used, then builds and stores the instance:

```
# hathor/conf/get_settings.py:83
    if not is_yaml:
        log = logger.new()
        log.warn(
            "Setting a config module via the 'HATHOR_CONFIG_FILE' env var will be deprecated soon
            "Use the '--config-yaml' CLI option or the 'HATHOR_CONFIG_YAML' env var to set a yaml
        )
    from hathor.conf.settings import HathorSettings as Settings

    settings_loader = load_yaml_settings if is_yaml else load_module_settings
    _settings_singleton = _SettingsMetadata(
        source=source,
        is_yaml=is_yaml,
        settings=settings_loader(Settings, source)
    )
    return _settings_singleton.settings
```

`settings_loader` is chosen by the `is_yaml` flag — either `load_yaml_settings` or `load_module_settings`, both imported from `hathorlib.conf.utils` (`get_settings.py:22`). This is the **dispatch** idea from Chapter 4: pick the function from a flag, then call it uniformly. The YAML loader parses the file and feeds the resulting dict into the Pydantic model; the module loader reads constants off a Python module. Either way, the output is a validated `HathorSettings`, stored in the cache, and returned. Every subsequent call short-circuits at the `if _settings_singleton is not None` check above.

A companion accessor, `get_settings_source()` (`get_settings.py:62`), returns the path that was loaded — useful for logging "which network am I on" — and asserts that settings were loaded first.

The `HathorSettings` type — validate at load

Now the type itself, in `settings.py`. It is a Pydantic model — and this is the recent migration the chapter set out to verify: it **is** Pydantic (Pydantic v2's `ConfigDict`, `field_validator`, `model_validator`, imported at `settings.py:16`), built as a subclass of a base model that ships in the shared `hathorlib` library:

```
# hathor/conf/settings.py:32
class HathorSettings(LibSettings):
    model_config = ConfigDict(extra='forbid')
```

Two facts are packed into those two lines.

It inherits from `LibSettings`. `LibSettings` is `hathorlib.conf.settings.HathorSettings` (imported at `settings.py:22`), itself a Pydantic model. The bulk of the field declarations — the network name (`P2P_NETWORK_NAME: str`), the size limits, the timing constants, the genesis fields — live in that shared library base class, and `hathor-core`'s subclass adds the fields specific to a full node (checkpoints, feature activation, the consensus-algorithm choice). The reason for the split: `hathorlib` is reused by lighter Hathor tools

(wallets, libraries) that need the same constants without the full node; keeping the common settings in the library avoids two copies drifting apart — the single-source-of-truth principle applied across packages, not just within one.

`extra='forbid'`**.** This is a Pydantic configuration that makes the model **reject any field the YAML contains that the model does not declare.** A typo in a profile — `DECIMAL_PLACE` instead of `DECIMAL_PLACES` — does not silently fall through as an ignored extra key; it raises at load. This is the validation half of read-only's safety story: not only should nothing change the object after load, you cannot even load an object with unexpected contents. Combined with type validation, it means a malformed profile is caught at boot, not at the moment some component first reads a missing constant deep in the call stack. (This is the fail-fast vs. crash-deep trade-off: pay the cost of a clear error at startup, instead of an obscure one hours later.)

The subclass adds three full-node fields, each with a validator that turns raw YAML into the right Python type:

```
# hathor/conf/settings.py:36
    CHECKPOINTS: list[Checkpoint] = []

    @field_validator('CHECKPOINTS', mode='before')
    @classmethod
    def _parse_checkpoints(cls, checkpoints):
        # YAML gives {height: "hexhash"}; turn it into [Checkpoint(height, bytes), …]
        if isinstance(checkpoints, dict):
            return [Checkpoint(h, bytes.fromhex(_hash)) for h, _hash in checkpoints.items()]
        ...
```

A validator[6] is a method Pydantic runs while building the object. `mode='before'` means it runs before type-coercion, so it sees the raw YAML shape — here, a `{height: hexstring}` dict — and converts it into the typed shape the field declares — a `list[Checkpoint]`. This is how a human-friendly YAML representation (a map of heights to hex strings) becomes a strongly-typed list the rest of the code can rely on. The `FEATURE_ACTIVATION` field (`settings.py:54`) has the same shape: a sub-model built from a nested dict.

The third field carries a cross-field rule — the kind no scattered-globals design could enforce:

```
# hathor/conf/settings.py:65
    CONSENSUS_ALGORITHM: ConsensusSettings = PowSettings()

    @model_validator(mode='after')
    def _validate_consensus_algorithm(self) -> Self:
        """Validate that if Proof-of-Authority is enabled, block rewards must not be set."""
        if self.CONSENSUS_ALGORITHM.is_pow():
            return self
        if (self.BLOCKS_PER_HALVING is not None or
                self.INITIAL_TOKEN_UNITS_PER_BLOCK != 0 or
                self.MINIMUM_TOKEN_UNITS_PER_BLOCK != 0):
            raise ValueError('PoA networks do not support block rewards')
        return self
```

A `model_validator(mode='after')` runs once, after all individual fields are set, so it can see the whole object and check relationships between fields. The rule: a proof-of-authority[4] network mints no block reward (in PoA, blocks are signed by authorities, not mined, so there is nothing to reward). If a profile declares PoA and a reward schedule, the two are contradictory, and the model refuses to build. This is the payoff of the single-validated-object design from §22.2: cross-field invariants live in exactly one place and are checked exactly once, at load.

The module also defines a few monetary constants used to compute genesis amounts (`settings.py:24`):

```
# hathor/conf/settings.py:24
DECIMAL_PLACES = 2                                  # HTR has 2 decimal places
GENESIS_TOKEN_UNITS = 1 * (10 ** 9)                 # 1B units
GENESIS_TOKENS = GENESIS_TOKEN_UNITS * (10 ** DECIMAL_PLACES)  # → 100B (with decimals)
HATHOR_TOKEN_UID = b'\x00'                          # the native token's id is a single null byt
```

`HATHOR_TOKEN_UID = b'\x00'` is worth a glance: the native token HTR is identified everywhere in the code by a single null byte, distinguishing it from custom tokens (which get longer ids). That one constant ripples through the entire monetary subsystem.

A representative profile excerpt

The default profile is a YAML file. A profile reads as a flat list of the constants the model declares; below is a representative excerpt (field names are real `HathorSettings` fields; treat the exact values as illustrative of shape, not as line-verified mainnet numbers):

```
# (representative excerpt of a YAML profile)
P2P_NETWORK_NAME: mainnet              # announced in the peer handshake
DECIMAL_PLACES: 2                      # HTR has two decimal places

# genesis — the shared root of history; differs per network
GENESIS_BLOCK_HASH: "00000033...e9c"
GENESIS_BLOCK_NONCE: 3526202
GENESIS_OUTPUT_SCRIPT: "76a914...88ac"

# monetary schedule
INITIAL_TOKEN_UNITS_PER_BLOCK: 64      # block reward when the chain starts
MINIMUM_TOKEN_UNITS_PER_BLOCK: 1       # reward floor after halvings
BLOCKS_PER_HALVING: 2102400            # how often the reward halves

# timing / difficulty
AVG_TIME_BETWEEN_BLOCKS: 30            # target seconds per block

# structural limits
MAX_NUM_INPUTS: 255
MAX_NUM_OUTPUTS: 255
```

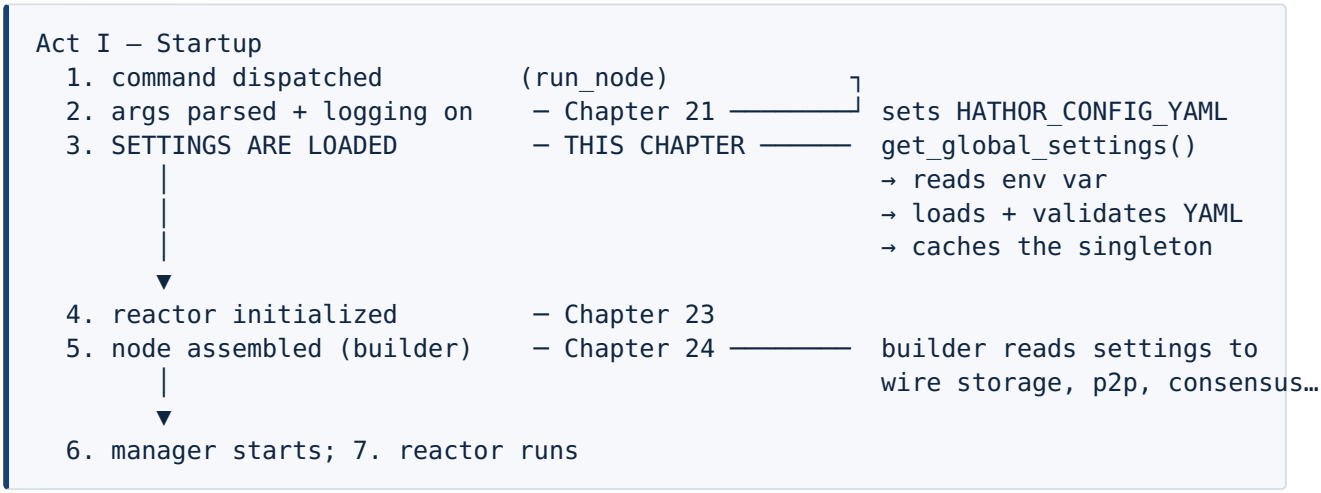
Walk three of these to feel why a mismatch is fatal:

- **`P2P_NETWORK_NAME`.** Two nodes compare this string during the handshake (Ch. 34). If yours says `mainnet` and the peer's says `testnet-golf`, the handshake fails and you never connect. A one-word difference is the boundary between two networks.
- **`GENESIS_BLOCK_HASH` (and the rest of the genesis data).** This is the root of history. If your genesis bytes differ from the network's by a single bit, your hash is different, your chain has a different bottom, and no block the network produces will ever link to yours. You will reject everything and be rejected by everything — silently.
- **`AVG_TIME_BETWEEN_BLOCKS`.** This feeds the difficulty-adjustment math (Ch. 32). Get it wrong and your node computes a different expected weight for blocks than the network does, so it disagrees about which blocks are valid. Again: not a crash — a fork.

That is the whole reason this package validates so strictly and treats its output as read-only. The cost of a wrong constant is not an exception you can catch; it is a node that looks healthy and is on its own private island.

22.5 How it plugs into the lifecycle

Place this chapter precisely in the boot story from Chapter 0, §0.3, **Act I**:



The sequence matters. Settings load **after** the command line (Ch. 21), because the chosen network comes from a flag the CLI translates into the env var. Settings load **before** the builder (Ch. 24), because the builder needs the constants to construct components correctly — it reads the network name to set up p2p, the genesis to seed storage, the consensus algorithm to pick a consensus engine, and so on. From step 5 onward, every component that needs a constant calls `get_global_settings()` and receives the one cached, validated, read-only object. By the time the reactor is running (step 7), the settings have been fixed and shared for the entire rest of the process's life — they are never reloaded.

One design consequence to carry forward: because the value is a process-wide singleton selected by an env var, **tests and the simulator** (Ch. 43) must set that env var (or otherwise install a settings object) before anything reads it, and cannot freely switch networks mid-process — the duplicate-load guard in §22.4 will stop them. That constraint is the price of the single-source-of-truth guarantee, and the test infrastructure is built around it.

Recap

Question	Answer	Where in code
Which network am I on?	Chosen by an env var the CLI sets (`HATHOR_CONFIG_YAML`), default `mainnet`	`get_settings.py:43`, `:58`
How is a profile stored?	A **YAML** file of constants (legacy Python-module path is deprecated)	`get_settings.py:53–59`
What turns YAML into an object?	A **Pydantic** model with field/model validators	`settings.py:32`, `:36`, `:65`

Question	Answer	Where in code
Why won't a typo slip through?	`model_config = ConfigDict(extra='forbid')` rejects unknown fields	`settings.py:33`
Why only one settings object?	A module-level **singleton** cache, with a guard against loading two	`get_settings.py:36`, `:72`
How does everyone read it?	One accessor, `get_global_settings()`, returns the cached instance	`get_settings.py:39`
Why does a wrong constant matter?	A node with different constants is on a different network, silently	§22.2, §22.4

Settings is the node's single source of truth for "what network am I on and what are its rules." It is loaded once, validated strictly at load (fail fast, not crash deep), cached as a read-only singleton, and read by everything below it. You now know how `get_global_settings()` chooses a profile from the environment, parses it into a `HathorSettings` with Pydantic, and refuses to load two networks at once. The next chapter (23) covers the reactor abstraction the node spins up immediately after settings are fixed; from there, Chapter 24 takes this settings object into the builder, where it is read to wire the entire node together.

---

1. The genesis is the hard-coded first block and initial transactions that every node on a network agrees on by definition — the shared root of history. Different networks have different genesis data; that difference is part of what makes them distinct networks. Full treatment in Ch. 25 & 32. ↩
2. Weight is a numeric measure of how much proof-of-work a vertex represents; consensus prefers the history with the most accumulated weight. The constants that drive the weight/difficulty math live in the settings profile. Full treatment in Ch. 9 & 32. ↩
3. Feature activation is Hathor's mechanism for switching protocol upgrades on over a schedule, by miner signalling. The schedule for a network is part of its settings profile (the `FEATURE_ACTIVATION` field). Full treatment in Ch. 38. ↩
4. Proof-of-authority (PoA) is a consensus variant used by certain private networks, where blocks are signed by designated authorities rather than mined via proof-of-work. Because nothing is "mined," PoA networks define no block reward — a rule the settings model enforces. Full treatment in Ch. 32. ↩↩
5. A read-only object is one that nothing is supposed to mutate after it is created; configuration is treated this way so no stray code can change a network constant mid-run. (Pydantic models are mutable by default unless explicitly frozen; in Hathor the read-only discipline is enforced partly by convention and partly by the load-time `extra='forbid'` validation that rejects malformed input in the first place.) ↩

6. A validator (in Pydantic) is a method that runs while the model is being built, to coerce raw input into the declared type (`mode='before'`) or to check relationships between fields once they are all set (`mode='after'`). It is how a human-friendly YAML shape becomes a strongly-typed, internally-consistent object — or is rejected. ↩
7. An environment variable is a named value the operating system makes available to a running process (read in Python via `os.environ`). It is a process-global channel: any part of the program can read it without being passed it explicitly, which is why it suits a setting that must be reachable from hundreds of call sites. ↩